```
import numpy as np
import pandas as pd
import  matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error,r2_score,accuracy_score,confusion_matrix,classification_report
sns.set(style="whitegrid")
```

```
df=pd.read_csv("insurance.csv")
df.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

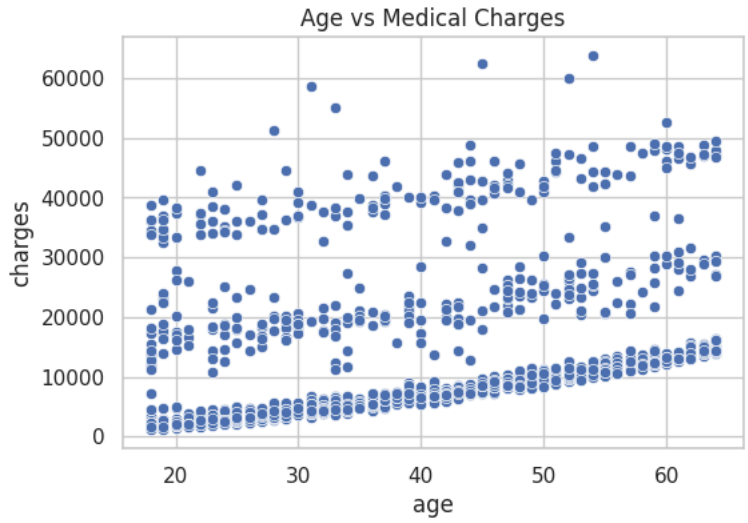
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
df.describe()
```

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

```
# Visualizing relationship with charges
plt.figure(figsize=(6,4))
sns.scatterplot(x='age', y='charges', data=df)
plt.title("Age vs Medical Charges")
plt.show()
```



Data preprocssing@convert categorical variable using one-hot encoding

```
df_encoded=pd.get_dummies(df,drop_first=True)
df_encoded.head()
```

	age	bmi	children	charges	sex_male	smoker_yes	region_northwest	region_southeast	region_southwest
0	19	27.900	0	16884.92400	False	True	False	False	True
1	18	33.770	1	1725.55230	True	False	False	True	False
2	28	33.000	3	4449.46200	True	False	False	True	False
3	33	22.705	0	21984.47061	True	False	True	False	False
4	32	28.880	0	3866.85520	True	False	True	False	False

```
# Features & Target
X = df_encoded.drop("charges", axis=1)
y = df_encoded["charges"]

# Train-test split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
#train linear regression model
lr=LinearRegression()
lr.fit(X_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

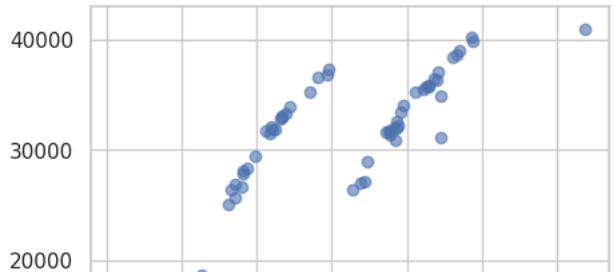
```
# Predictions & Evaluation
y_pred = lr.predict(X_test)

from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MSE: 33596915.85136145
RMSE: 5796.284659276273
R2 Score: 0.7835929767120724
```

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Predictions
y_train_pred = lr.predict(X_train)
y_test_pred = lr.predict(X_test)

# RMSE calculations
```

```
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))

# R2
train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("MODEL PERFORMANCE SUMMARY")
print("-" * 40)

print(f"Train RMSE : {train_rmse :.2f}")
print(f"Test RMSE : {test_rmse :.2f}")
print(f"Train R2:  { train_r2:.3f}")
print(f"Test R2: {train_r2:.3f}")

print("\nINTERPRETATION")
print("-" * 40)

print(f"On average, the model makes an error of about {test_rmse :. 2f} units in predicting medical charges.")

# Overfitting / Underfitting diagnosis
if test_rmse > train_rmse * 1.3:
  print("A Possible OVERFITTING detected:")
  print(" - Model performs much better on training data than test data.")
elif test_rmse < train_rmse * 1.1:
  print("Model generalizes well:")
  print("- Training and testing errors are similar.")
else:
  print("A Possible UNDERFITTING:")
  print(" - Model may be too simple to capture patterns.")
```

```
MODEL PERFORMANCE SUMMARY
----------------------------------------
Train RMSE : 6105.55
Test RMSE : 5796.28
Train R2:  0.742
Test R2: 0.742

INTERPRETATION
----------------------------------------
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-472305022.py in <cell line: 0>()
     25 print("-" * 40)
     26
---> 27 print(f"On average, the model makes an error of about {test_rmse :. 2f} units in predicting medical charges.")
     28
     29 # Overfitting / Underfitting diagnosis

ValueError: Format specifier missing precision
```

```
plt.figure(figsize=(5,5))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
          linestyle=' -- ')
plt.xlabel("Actual Charges")
plt.ylabel("Predicted Charges")
plt.title("Linear Regression: Actual vs Predicted")
plt.show()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-3037377289.py in <cell line: 0>()
      1 plt.figure(figsize=(5,5))
      2 plt.scatter(y_test, y_pred, alpha=0.6)
----> 3 plt.plot([y_test.min(), y_test.max()],
      4          [y_test.min(), y_test.max()],
      5           linestyle=' -- ')

                              8 frames
/usr/local/lib/python3.12/dist-packages/matplotlib/_api/__init__.py in check_in_list(values, _print_supported_values,
**kwargs)
    128             if _print_supported_values:
    129                 msg += f"; supported values are {', '.join(map(repr, values))}"
--> 130             raise ValueError(msg)
    131
    132

ValueError: ' -- ' is not a valid value for ls; supported values are '-', '--', '-.', ':', 'None', ' ', '', 'solid',
'dashed', 'dashdot', 'dotted'
```